DHT Construction over UDP

CSE 434 Socket Project Milestone Design Document

Group 71

Purpose. This document describes the UDP message formats, DHT setup sequence, record distribution, validation, and milestone demonstration evidence.

This milestone implements an always-on manager and an interactive peer. The manager tracks membership; the peers build a directed logical ring and distribute the 14-field storm records into local hash tables.

Submission identity

Group: 71 Contributors: Rohan Reddy Boddu, Swayam Vinod Sugandhi Deadline: September 27, 2026, before 11:59 p.m.

Milestone boundary

Implemented manager commands: register, setup-dht, dht-complete. Peer messages: set-id, store, stored and count-ring. The last two support reliable completion and reporting. Querying, churn, teardown and graceful deregistration are reserved for the full project.

Process and socket model

The manager accepts one argument: its UDP port. Each peer accepts two: the manager IPv4 address and UDP port. A peer binds its advertised IPv4 address to one m-port for manager exchanges and one p-port for peer exchanges. Commands come from standard input. DHT_GROUP supplies the assigned port range without changing these positional interfaces. The peer main thread reads commands and waits for manager responses. A separate receiver thread services peer traffic continuously. It never waits for another peer while handling a datagram. The manager processes one datagram at a time, so membership changes are serialized.

Current evidence status

Group 71 is configured. Eight local tests pass, including a four-process UDP run and induced packet loss. The 1950 dataset has also passed a local three-peer run. The two-host demonstration, video upload and actual video timestamps remain pending. Basis: supplied Socket Project Fall 2026.pdf, sections 1.1, 1.2.1, 2, 3 and 4.1; Canvas assignment "Socket Project: Milestone", assignment 7832035.

Message formats: manager

One UTF-8 JSON object occupies one UDP datagram, with a maximum of 8,192 encoded bytes. JSON escapes strings; no delimiter escaping is required at the application layer. Object key order is not meaningful.

Common envelope

v: integer (=1); command: string; request_id: 32 lowercase hex characters Each new request uses a UUID-derived request ID. Retries retain the exact request and ID. Responses copy the ID, use command="response", identify for_command, and contain code="SUCCESS" or "FAILURE". Failures contain reason:string.

register

peer_name: string; ip: IPv4 string; m_port: integer; p_port: integer Sent from the peer m-port to the manager port. Names contain 1-15 ASCII letters. All ports are globally distinct and inside the group range. The source endpoint must match ip/m_port. On success, return state="Free". Duplicate names, ports, invalid values or a full registry return FAILURE with no mutation.

setup-dht

peer_name: string; n: integer [3,32]; year: integer [1950,2019] Only a registered source can initiate construction. Require no existing DHT and at least n free peers. Select n-1 other peers randomly; mark the requester Leader and the selected peers InDHT. SUCCESS adds peers:[[name,ip,p_port], ...] with leader first, and dht_id:string equal to the setup request ID. The manager enters BUILDING.

dht-complete

peer_name: string Sent from the registered leader m-port after all stores and counts are verified. Only the leader of a BUILDING DHT can succeed; SUCCESS adds phase="READY". Other senders or phases fail. New non-completion commands fail while BUILDING. Exact retransmissions are answered from the cache.

Manager state transitions

EMPTY --successful setup-dht--> BUILDING --leader dht-complete--> READY A second setup fails in BUILDING or READY. Registration normally adds Free peers; while BUILDING, new registrations fail. Malformed envelopes are logged and dropped.

Message formats: peer ring

Every peer request also has the common envelope. dht_id identifies the build session; after initialization, a peer accepts ring messages only from its recorded left neighbor and for the current session.

set-id

dht_id: string; id: integer; n: integer; peers: tuple list; s: integer; total: integer; year: integer Leader sends directly to each selected peer before data distribution. The peer validates membership, sets id and right neighbor, and allocates s buckets. SUCCESS/FAILURE uses the common response format. Repeating the same configuration does not reset the table; conflicting configurations fail.

store

dht_id: string; seq: integer; pos: integer; owner: integer; record: array of 14 strings The leader sends only to its right neighbor; intermediate peers recompute both hashes and forward to their right neighbor. The owner stores once. seq is the zero-based row index. A store arriving back at ID 0 is an error.

stored

dht_id: string; seq: integer; pos: integer; owner: integer Owner copies the store request_id and these routing fields, removes the record, and sends this receipt clockwise. Intermediate peers forward it; the leader matches its pending store. There is no reverse application acknowledgement.

count-ring

dht_id: string; counts: array of [id:int, name:string, count:int] Leader starts counts with its own count. Each peer appends its ID, name and stored count. The token returns clockwise to ID 0 with the same request_id. The leader validates each count against the distribution calculated from the CSV before signaling completion.

record field order

event_id, state, year, month_name, event_type, cz_type, cz_name, injuries_direct, injuries_indirect, deaths_direct, deaths_indirect, damage_property, damage_crops, tor_f_scale. Values are preserved as strings; numeric event IDs are used for hashing. Quoted CSV commas and empty fields are preserved.

Registration and ring selection

Figure 1. register

Peer m-port Manager Read command; bind m-port and p-port

register(name, ip, m_port, p_port) Validate uniqueness, ports and source Insert peer with state Free response: SUCCESS, state=Free

Start p-port receiver; accept next command

Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Figure 2. setup-dht

Leader m-port Manager Preflight CSV, prime size and packet bounds setup-dht(name, n, year) Validate; randomly choose n-1 Free peers Set Leader/InDHT; phase=BUILDING SUCCESS, leader-first peer tuples, dht_id Begin set-id, stores and count-ring

Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Failure paths: the manager returns FAILURE with a reason and does not perform the requested transition. The peer prints COMMAND_FAILED. While BUILDING, only a new valid dht- complete can transition the manager to READY.

Initialization and completion

Figure 3. set-id (repeat for i = 1 to n-1)

Leader p-port Peer i p-port Initialize own ID 0 and local table set-id(i, n, tuples, s, total, year, dht_id) Validate source and configuration Store ID, right neighbor and empty table

response: SUCCESS for set-id Wait for all peers before first store

Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Figure 4. dht-complete

Leader m-port Manager All receipts received; counts verified Print each peer count and total dht-complete(leader name) Validate registered source and leader Transition BUILDING to READY response: SUCCESS, phase=READY

Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Set-id is the initialization exception to clockwise routing. It is completed before any store is issued. The application automatically sends dht-complete after successful verification; stdin cannot force premature completion.

Record placement and receipts

Figure 5. store (example: record belongs to ID 2)

Leader / ID 0 Peer / ID 1

Owner / ID 2 pos = event_id mod s; owner = pos mod n

store(record, pos, owner=2) Recompute hash; ID is not owner store: forward unchanged

Insert in bucket[pos]; deduplicate Stop record; create stored receipt

Time flows downward; arrows are UDP messages; shaded boxes are local actions.

Figure 6. stored (example: owner is ID 1)

Leader / ID 0 Owner / ID 1 Peer / ID 2 Record stored; retain request_id stored(seq, pos, owner=1) Forward receipt to right neighbor stored: same request_id Match pending store and routing fields

Proceed to next CSV record

Time flows downward; arrows are UDP messages; shaded boxes are local actions.

For an ID-0-owned record, the leader inserts locally without a store message. For an ID-2-owned record in a three-node ring, its receipt travels directly on the ring edge 2 to 0. The record itself never returns to the leader.

Counting and loss recovery

Figure 7. count-ring

Leader / ID 0 Peer / ID 1 Peer / ID 2 Seed counts with [0, name,

local_count] count-ring([count0]) Append [1, name, local_count] count-ring([count0,count1]) Append [2, name, local_count] count-ring(all three counts)

Verify counts, print, then

dht-complete

Time flows downward; arrows are UDP messages; shaded boxes are local actions.

UDP retransmission policy

The coordinator waits 0.5 seconds for a matching response or complete ring receipt and retries the exact request up to 10 sends. Manager operations and set-id use direct paired replies. Store and count-ring responses follow the ring, preserving its direction. A lost receipt causes another store traversal, not another stored record. Manager responses are cached by (source endpoint, request_id), along with the original request bytes. Reusing an ID for a different request fails. The bounded cache retains 4,096 responses. Its guarantee covers retained requests during this run; it is not durable across restarts. Set-id is idempotent for identical configuration. Local buckets reject a second copy of the same event ID and reject conflicting contents. Count retries start with a fresh leader count and append one entry per peer. Delayed receipts are matched by ID or ignored after the corresponding pending request is removed.

Failure behavior

A persistent timeout leaves the build incomplete, and dht-complete is not sent. Restart the manager and all peers after a partial/uncertain build. There is no rollback or peer-crash recovery in the milestone.

Data structures and validation

Storage and complexity

The manager uses a dictionary keyed by peer name; entries hold IPv4, both ports and Free/Leader/InDHT state. It also holds a ring-name list, leader and phase. Registry lookup is expected O(1); checking port uniqueness and selecting peers costs O(P), where P is registered peers. Each peer stores an ordered tuple list indexed by ring ID, a configuration/session tuple, and an explicit array of s buckets. The right neighbor is tuples[(id+1) mod n]. Separate chaining preserves distinct events that collide at pos; duplicate checks scan only that bucket. For l CSV records, s is the first prime strictly greater than 2*l; pos=event_id mod s and owner=pos mod n. Each peer allocates O(s) bucket headers and O(l_i) stored records; the leader additionally holds O(l) input records for preflight and verification. The ring costs O(n) membership space per peer. Local insertion is O(1+b) for bucket size b, worst-case O(l). One nonlocal record causes exactly n clockwise transmissions in the loss-free case: store messages to its owner and receipts for the remaining path. Thus distribution costs O(l*n) messages, plus initialization and the count pass. Stop-and-wait favors clarity and bounded in-flight state over throughput.

Input and resource decisions

The parser skips one normalized 14-column header, uses csv.reader for quoted fields, and validates year, numeric event IDs and nonnegative injury/death counts. It rejects duplicate event IDs and empty datasets. Bounds are 32 peers, 100,000 records and 8,192 bytes per datagram. Records are serialized in preflight before setup locks the manager. Both ports bind to the advertised local IPv4 interface. Global port uniqueness follows the assignment conservatively even for peers on different

hosts. The group range is [1000+500*G, 1499+500*G]. Peers must be directly reachable; NAT traversal is not included. Session/source checks detect misrouting but do not provide cryptographic authentication.

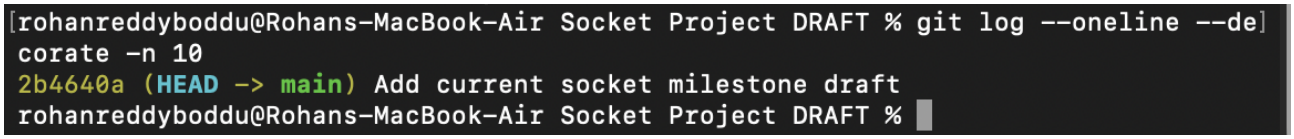
Observed tests and remaining validation

Eight tests passed: input/schema checks, CSV edge cases, collision preservation, manager rejection/state transitions, retry idempotence and two four-process UDP tests (normal and induced packet loss). The synthetic 40-record fixture yields s=83 and counts [13,14,13]. Actual transmissions were checked against each sender's right neighbor. A separate local run with the 223- record 1950 data reached READY with counts 72, 75 and 76. The two-host demo remains unverified.

Development and demo evidence

Version-control snapshot

The image below is generated from the current git log in this project folder. It records the project snapshot used for this milestone submission.



Video link and actual timestamps

Video Link: https://youtu.be/MHJa6y49GAk

3(a) Not applicable — Python is interpreted; no separate compilation step is shown.

3(b) 0:00–1:45 — Programs are shown on general6 and general7.

3(c) 1:45–5:55 — Manager starts and Alpha, Beta, and Gamma register.

3(d) 5:55–8:21 — Alpha runs setup-dht Alpha 3 1950; the ring is built and the final record counts and dht-complete are shown.

Full project additions

CSE 434 Socket Project | Group 71

Contributors: Rohan Reddy Boddu and Swayam Vinod Sugandhi

The preceding eight pages preserve the milestone document. The following sections extend that design to the full project. Earlier milestone scope, pending-demo statements, test counts and video timestamps describe the milestone snapshot; they are not final-project evidence. Where message formats or retry behavior differ, the definitions below describe the final implementation.

Scope and process changes

The final implementation adds query-dht, leave-dht, join-dht, dht-rebuilt, teardown-dht, teardown-complete and deregister. The manager and peer command-line arguments, UDP sockets and 14-field record format remain unchanged. Group 71 uses ports 36500-36999; the manager uses 36500.

The main peer thread handles stdin and manager exchanges. The receiver thread continues servicing peer traffic. Rebuild requests run in worker threads so a leader can distribute records while its receiver handles incoming messages. All peer hosts require the same year dataset in data/, or the directory selected by DHT_DATA_DIR.

Manager state transitions

Phase	Transition
EMPTY	setup-dht starts BUILDING.
BUILDING	Leader dht-complete makes the DHT READY.
READY	A valid leave or join starts REBUILDING; leader teardown starts TEARING_DOWN.
REBUILDING	The initiating peer sends dht-rebuilt; the manager commits the proposed membership and returns READY.
TEARING_DOWN	Leader teardown-complete frees all members and returns EMPTY.

During BUILDING, REBUILDING and TEARING_DOWN, new commands other than the matching completion fail. Cached exact retries remain answerable. Query and join require a Free peer; leave requires a ring member. The last member must use teardown rather than leave.

Final message formats: manager

Every datagram is a UTF-8 JSON object: v=1, command:string, request_id:32 lowercase hex characters. Responses contain the same request_id, command=response, for_command, code=SUCCESS or FAILURE, and reason on failure. The maximum encoded size is 8,192 bytes. New requests use UUIDs; retries reuse the same request ID and payload. The table lists fields in addition to the common envelope.

register

peer_name, ip, m_port, p_port state=Free; validate names, group ports, uniqueness and sender endpoint.

setup-dht

peer_name, n, year peers:[name,IP,p-port] tuples, leader first; dht_id. Select n-1 Free peers randomly.

dht-complete

peer_name phase=READY; accepted only from the building leader.

query-dht

peer_name peer: random entry tuple; dht_id; peers: ring tuples used to validate direct result senders.

leave-dht

peer_name old_peers, peers (proposed ring), year, dht_id. The clockwise successor becomes leader.

join-dht

peer_name old_peers, peers (old ring plus joining peer), year, dht_id. Current leader remains.

dht-rebuilt

peer_name, new_leader phase=READY, leader. Initiator and proposed leader must match pending change.

teardown-dht

peer_name SUCCESS; manager enters TEARING_DOWN; only ready leader allowed.

teardown-complete

peer_name phase=EMPTY; mark prior members Free and clear DHT state.

deregister

peer_name SUCCESS; remove a Free peer; peer exits after reply.

Non-register requests must originate from the registered m-port and IP for peer_name. While gated, non-completion commands fail. Exact retries receive cached responses before gate checks. A Leader cannot deregister until teardown or departure makes it Free. Initial setup requires n >= 3; departures may reduce the ring to 2 or 1. The last member must teardown instead of leaving an empty ring.

Final message formats: peers

A configuration contains peers, year, total, s, digest and dht_id. digest is SHA-256 of canonical JSON rows, allowing identical CSV data to be checked across hosts. A new setup or rebuild uses a new session ID.

set-id

Configuration. Each recipient derives its ID from the ordered membership list; direct response acknowledges initialization.

store

dht_id, seq, pos, owner, record:14 strings. Move clockwise until owner; do not return the store record to leader.

stored

dht_id, seq, pos, owner. Small receipt continues clockwise from owner to leader, replacing the store payload.

preflight

route, hop, year, total, digest, errors. Every non-origin participant checks its local CSV and appends errors.

count-ring

route, hop, dht_id, counts:[[id,name,count]]. Each node appends its actual local count.

teardown

route, hop, dht_id, final. Delete each local table. final=true also clears local membership.

reset-id

route, hop, config. Install proposed membership, assign new IDs and allocate empty tables.

rebuild-dht

initiator:[name,IP,p-port]. New leader repopulates the ring and returns response only after counts verify.

join-ring

peers, dht_id. Joining peer asks current leader to coordinate preflight, teardown, reset and rebuilding.

find-event

event_id, requester:[name,IP,p-port], dht_id, id_seq. Each probe appends its ID and randomly selects an unvisited node.

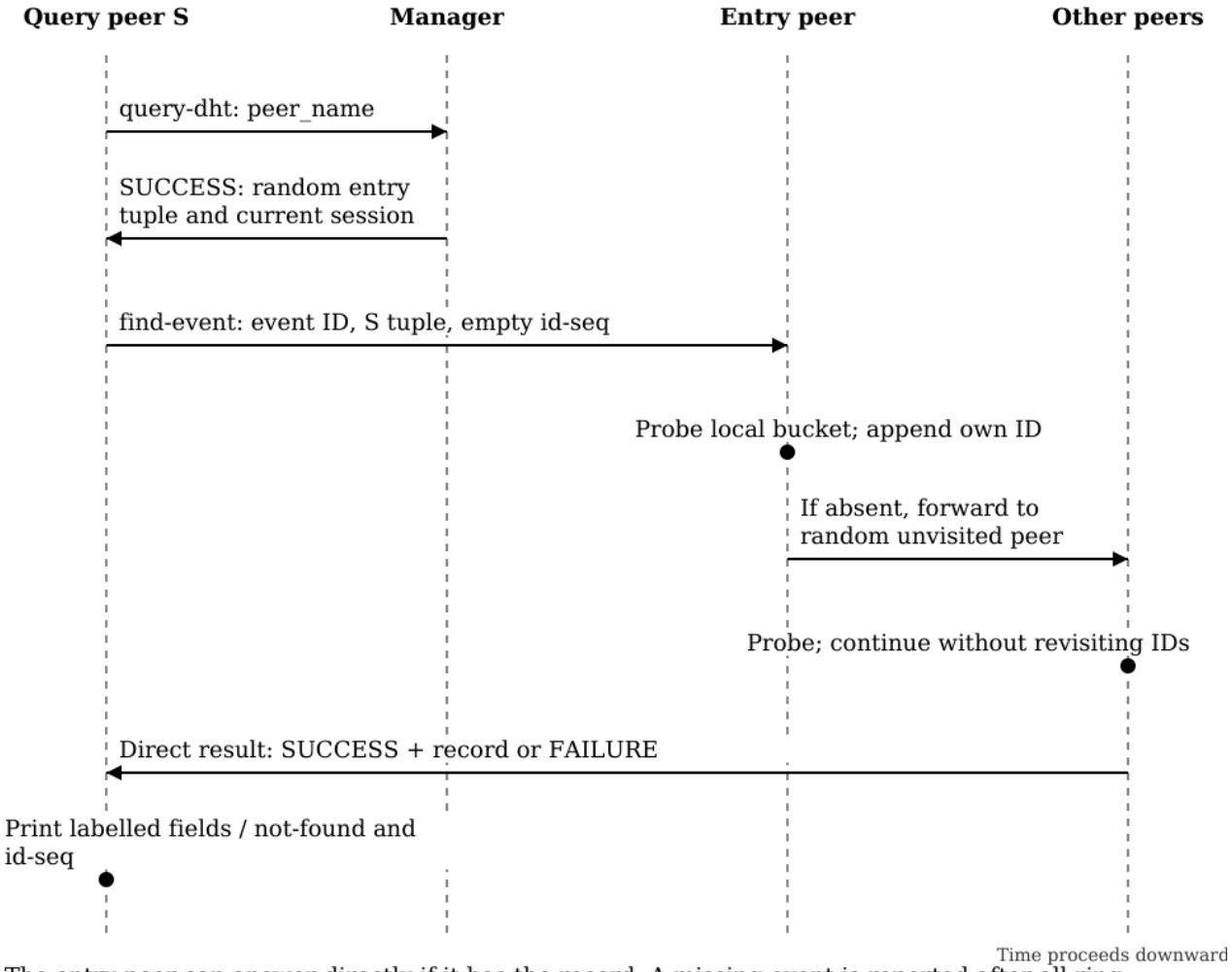
response for find-event

result:{code,record,id_seq,event_id}. Sent directly to requester. Outer code reports successful protocol handling; inner code reports found/not found.

Management tokens carry route beginning with their initiator, and hop beginning at 1. Each recipient checks the sender against the preceding route entry, processes once and increments hop. The final hop returns to the origin. Teardown/reset replay caches resend the saved outgoing token without repeating the mutation. The initial set-id exchange is the assignment-permitted direct initialization exception.

Hot potato queries

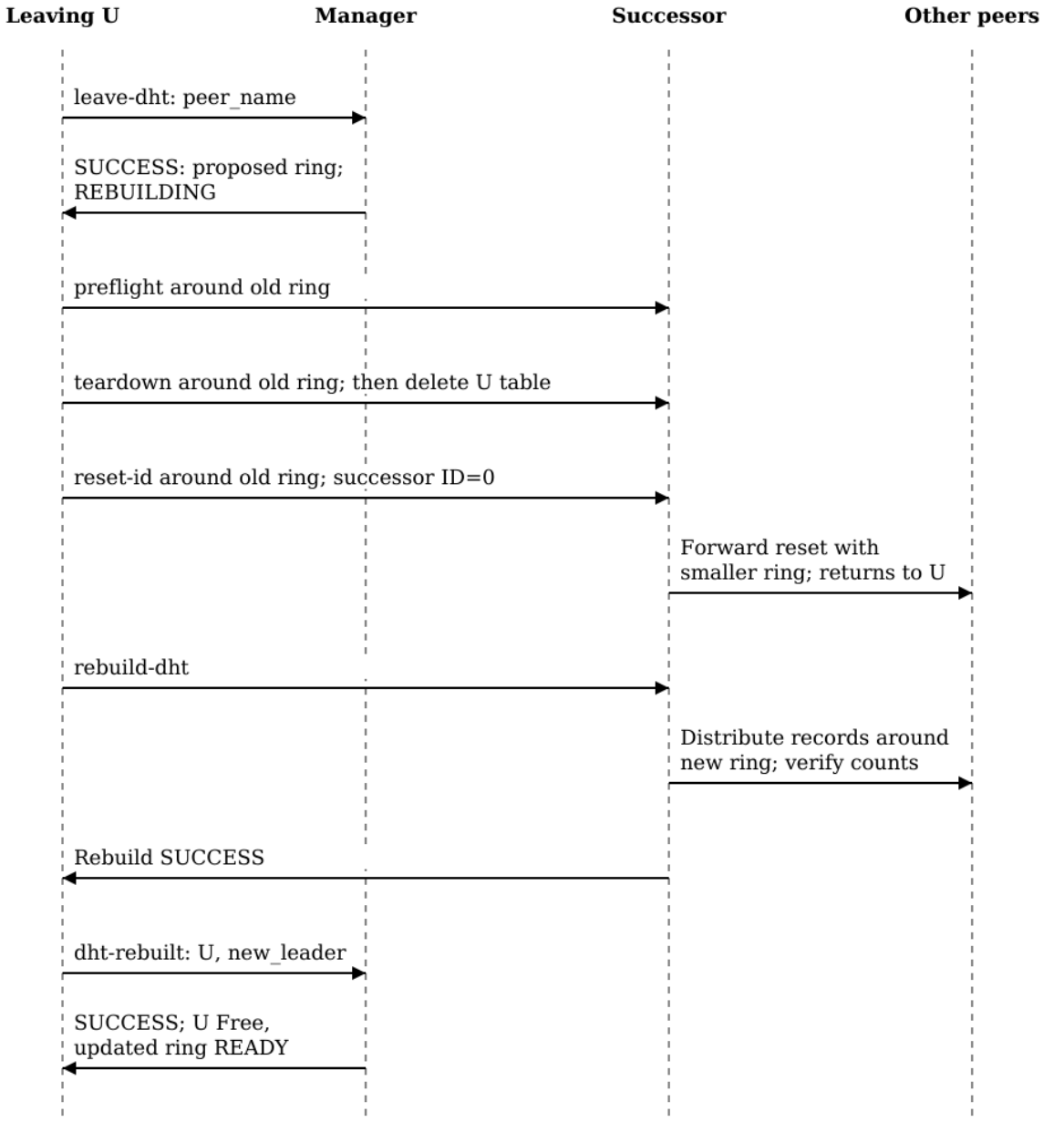
Only a registered Free peer can request query-dht while the manager is READY. The optional event ID in the CLI is convenience syntax; the manager still receives query-dht with peer_name only. The two-stage query-dht followed by find-event is also supported.



The entry peer can answer directly if it has the record. A missing event is reported after all ring members have been probed. Random visitation is tracked by id_seq; the remaining set is derived from ring IDs minus visited IDs. The responding node sends directly to S, not back around the ring. Retries can choose a different path; each returned path contains unique IDs.

Leave and rebuild

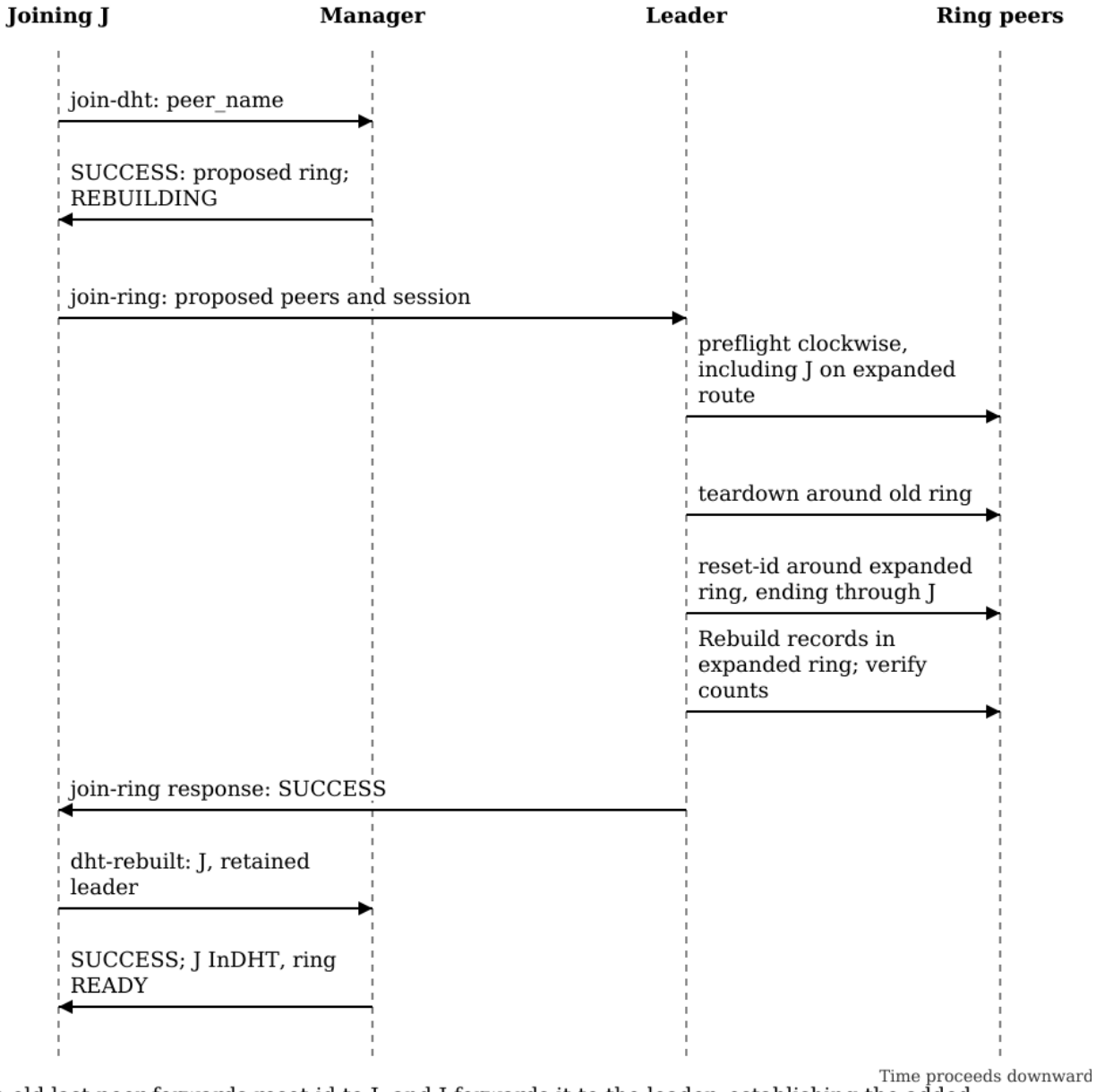
Peer U requests departure. The manager records U as the only allowed completion sender. The clockwise successor becomes the proposed leader; IDs are renumbered from that successor.



U retains its old routing context until the reset token returns and the rebuild finishes, then clears membership. It can issue query-dht after the manager acknowledges dht-rebuilt. The new leader reads the same year CSV from its own data directory.

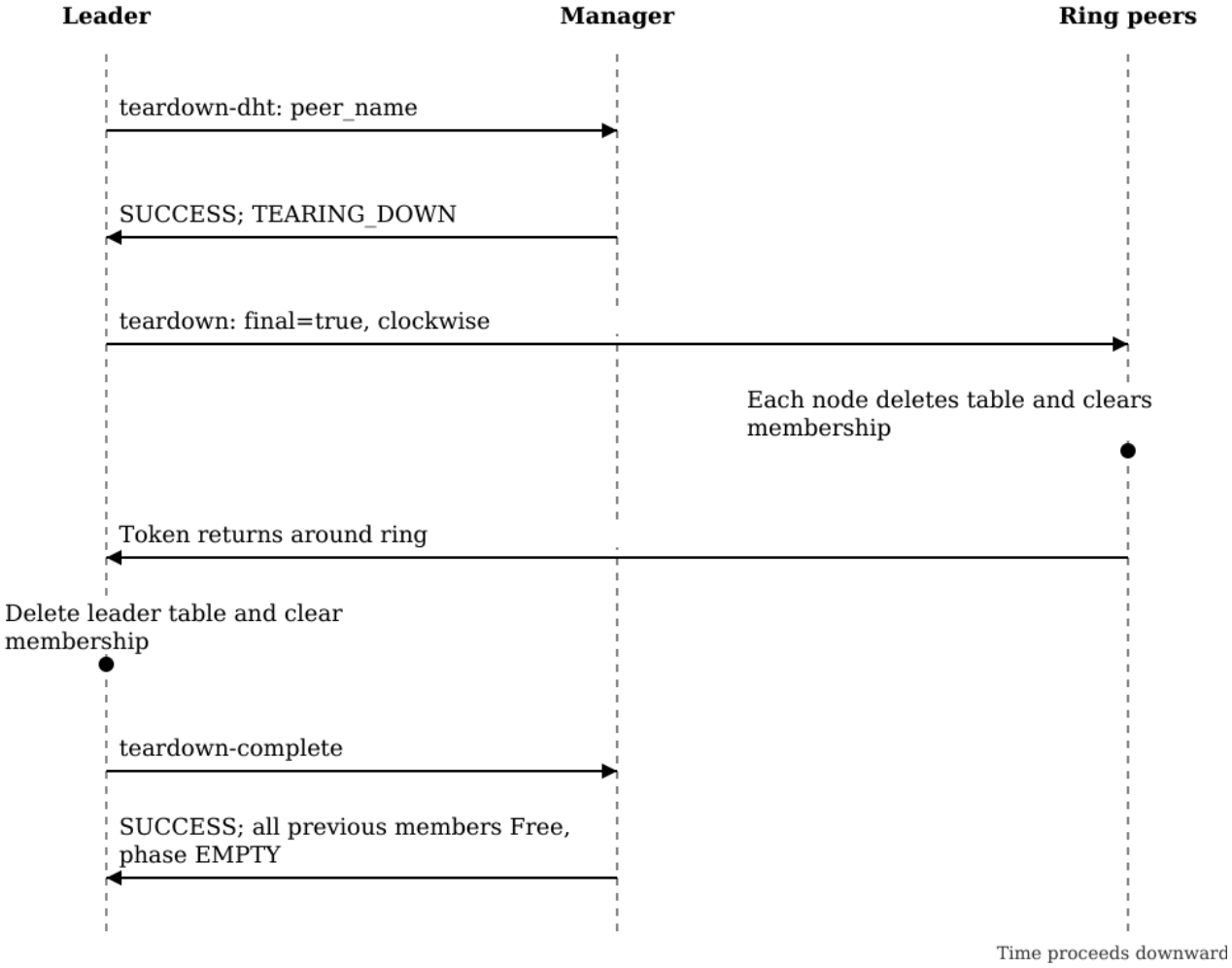
Join and rebuild

A Free peer J requests entry. This implementation appends J after the old last peer, keeps the current leader and resets the enlarged ring before redistribution. A bootstrap join-ring request connects the outside peer to the current leader; established-ring management then circulates clockwise.



The old last peer forwards reset-id to J, and J forwards it to the leader, establishing the added clockwise link. Every node installs the new session. Cached rebuild jobs prevent request retries from starting duplicate reconstructions.

Final teardown and reliability



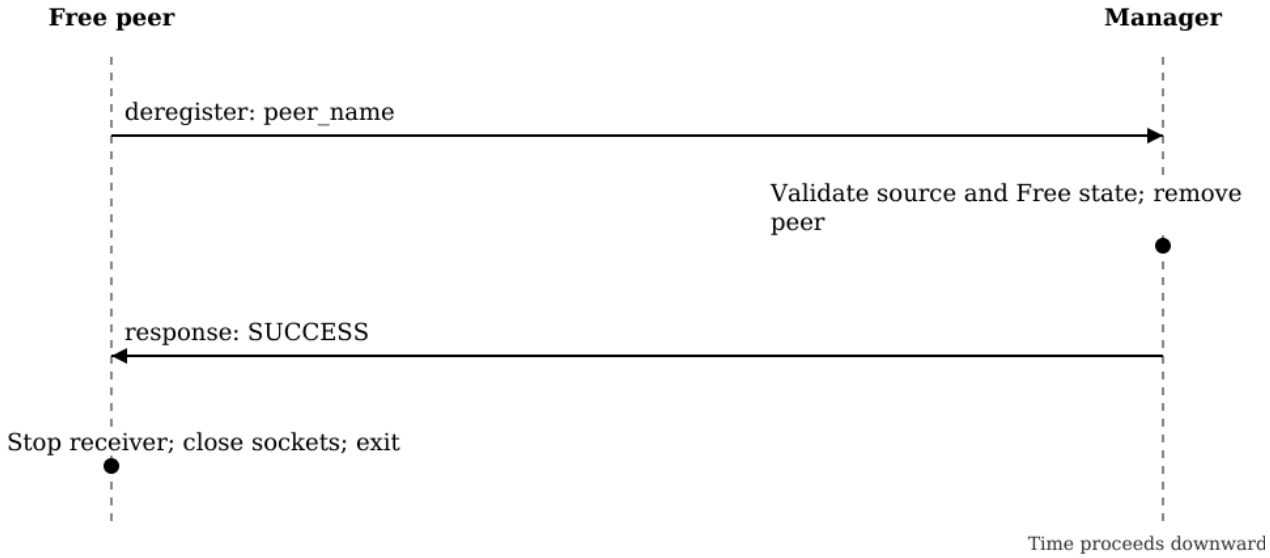
Failure handling and bounds

Manager retries last about 5 seconds, ordinary peer requests 5 seconds, control tokens 30 seconds, and rebuild jobs 600 seconds. Store deduplication and cached control forwarding tolerate transient loss. Completion is sent only after the requested work verifies. This design does not implement rollback or crash recovery: after an uncertain mutation failure, restart every process.

Capacity is 32 peers and 100,000 records. UDP datagrams are capped at 8,192 bytes; large datagrams can fragment. IPv4 must be mutually reachable. There is no authentication or NAT traversal. These choices target sequential demonstrations on a controlled course network.

Deregistration and added data structures

A peer can deregister only in the Free state. After final teardown, each former member is Free. The peer sends deregister from its registered m-port and waits for the reply before stopping its listener and closing its sockets. The manager is stopped explicitly after all peers exit.



Structures added for the full project

The manager keeps pending_change with the initiator, proposed ring and new session ID. Existing membership is committed only after the expected dht-rebuilt. Each peer keeps pending request queues keyed by request ID, including allowed sender endpoints and expected commands. A lock protects shared receiver state. A 2,048-entry control cache stores outgoing tokens and destinations; a job map prevents duplicate rebuild workers.

Queries carry id_seq, the ordered list of visited IDs. HashTable.find scans the selected bucket for the exact event ID. A dataset cache stores a file timestamp, size, record count and SHA-256 digest; it does not cache all records. Rebuilding rereads the CSV and verifies its digest.

Cost and design choices

For bucket size b, local lookup costs O(b). A missing query probes all n members. Each hop carries up to n visited IDs and computes the unvisited candidates, giving O(n squared) total routing work and visitation-list traffic in the worst case. A successful query may stop earlier. Query state requires O(n) space.

Leave and join rebuild all l records, costing O(l*n) distribution messages plus O(n) control-token hops without loss. Every peer reads the dataset for validation. Teardown and deregistration use O(n) and O(1) messages respectively without retries. Rebuilding the entire ring keeps ownership consistent when n changes, at the cost of pausing queries during redistribution.

Final validation and demonstration evidence

The supplied local terminal output shows three unit tests passing: collision lookup, manager lifecycle gates, and replay/source validation. The 1996 integration run passed with 48,534 records in 31.59 seconds using six peer subprocesses plus the manager on one host with loopback UDP. That run did not inject packet loss. These results do not establish a successful four-host final demonstration.

With the supplied 1996 dataset, event IDs 5536849 and 5539287 are present; 2402920 and 55770111 are absent. The hash-table size is 97,073. At n=5, the expected counts by node ID are 9,711, 9,691, 9,714, 9,704 and 9,714. Counts are assigned to IDs; peer names depend on random ring selection.

Final YouTube URL: https://youtu.be/lozhAbs3dog

Step	Required scene	Actual timestamp
3(a)	Python setup and syntax check.	0:15
3(b)	Programs on at least four distinct end-hosts.	0:25-1:20
3(c)	Start manager; register six peers.	1:20-3:55
3(d)	Build a five-peer DHT using 1996 data.	3:55-6:50
3(e)	Outside peer queries the four specified event IDs.	6:50-7:50
3(f)	A member leaves; that departed peer queries.	7:50-9:30
3(g)	One outside peer joins; the remaining outside peer queries.	9:30-12:30
3(h)	The current leader tears down the DHT.	12:30-13:00
3(i)	All peers deregister and exit; stop the manager.	13:00-13:45

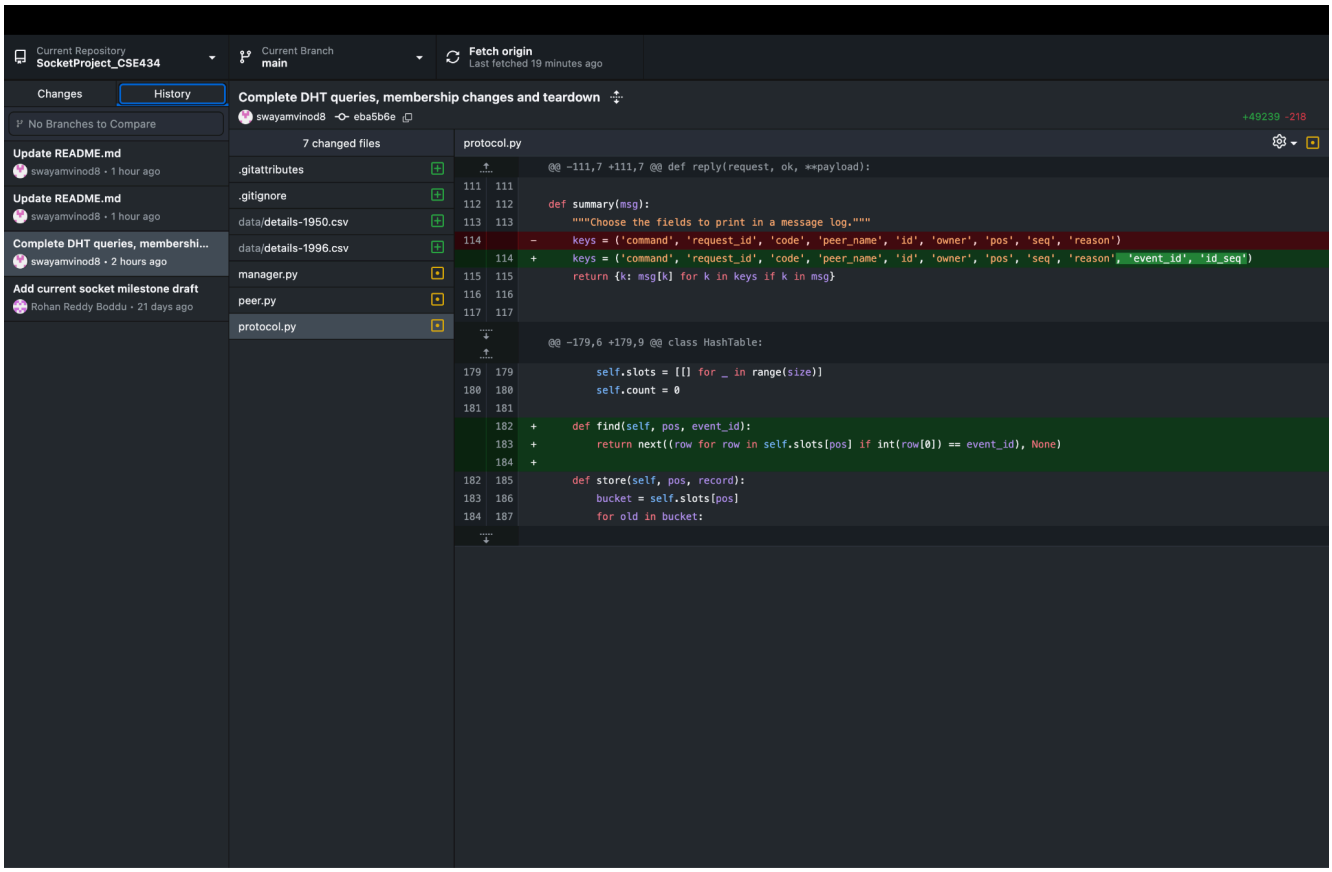
Record one continuous narrated video with readable traces, target length at most 15 minutes. Replace the pending fields with timestamps from the uploaded recording. The milestone video on page 8 is historical and does not replace the final demo.

Final version-control evidence

Git log:

```
[swayamvinod@Swayams-MacBook-Air SocketProject_CSE434 % git --no-pager log --pret]
ty=format:"%h - %an, %ad (Commit) - %cd (Author)"
1294f2c - swayamvinod8, Sun Oct 18 22:00:48 2026 -0700 (Commit) - Sun Oct 18 22:
00:48 2026 -0700 (Author)
d74c273 - swayamvinod8, Sun Oct 18 21:56:41 2026 -0700 (Commit) - Sun Oct 18 21:
56:41 2026 -0700 (Author)
eba5b6e - swayamvinod8, Sun Oct 18 21:13:03 2026 -0700 (Commit) - Sun Oct 18 21:
13:03 2026 -0700 (Author)
2b4640a - Rohan Reddy Boddu, Sun Sep 27 17:23:48 2026 -0700 (Commit) - Sun Sep 2
7 17:23:48 2026 -0700 (Author)%
```

Git history:



Reflog:

```
swayamvinod@Swayams-MacBook-Air SocketProject_CSE434 % git --no-pager reflog
1294f2c (HEAD -> main, origin/main, origin/HEAD) HEAD@{0}: commit: Update README
.md
d74c273 HEAD@{1}: commit: Update README.md
eba5b6e HEAD@{2}: commit: Complete DHT queries, membership changes and teardown
2b4640a HEAD@{3}: clone: from https://github.com/Rohanb1836/SocketProject_CSE434
.git
```

Integrity Check:



Google Drive:
https://drive.google.com/drive/folders/112QApc05yETQ-sSDKSZIPx3y7oNmBwoE?usp=share_link

GitHub:

https://github.com/Rohanb1836/SocketProject_CSE434.git